CLASHBOT: A Knowledge-Grounded LLM Agent That Plays Clash Royale in Real Time from Pixels

David Tyler Jackson (5298477)
University of Central Florida
CAP 6938 AI Knowledge Systems
Instructor: Dr. Ser Nam Lim

Abstract

We present CLASHBOT*, a non-embedded agent that plays the real-time card game Clash Royale on an iPhone through macOS screen mirroring, pixels in and taps out, with every model on one laptop. Perception and a learned play/wait gate feed a candidate generator built from a lane simulator, hybrid retrieval over 10,923 states from 197 pro games and a knowledge graph (840 nodes, 27,112 edges); a fine-tuned 600M-parameter LLM picks one of at most four candidates. On held-out games it picks the pro's play 50.7% of the time, against 25% chance and 38.4% for a placement prior. Zero-shot LLMs given the same prompt do far worse: Claude Sonnet 5.5 36.4%, Qwen3-14B 30.4%, Claude Haiku 5.5 27.9%. Three results are negative: a tuned LightGBM ranker (48.2%) statistically matches the LLM; corrupting the tool notes in its context leaves its accuracy unchanged; and answer rules mined from the pro games beat random but not card popularity. Open loop on 24 held-out pro games, the bot acts within 2 s of 77.1% of the pro's plays but picks the pro's card on only 17.1%, largely because that card is in its perceived hand only 40% of the time. A live decision takes 323 ms (median). The agent won one early Training Camp battle but lost all ten of a trial on frozen code (not significantly different from the prior agent KataCR's 1/20, Fisher $p = 1.0$), which its logs trace to misread tower HP, no committed target tower, weak spell valuation and over-playing.*

1. Introduction

Clash Royale is a real-time, two-player phone card game: each player brings a hidden eight-card deck, holds four cards, and spends elixir (regenerating up to ten) to place troops, buildings and spells on an 18×32-tile arena, for three minutes plus up to two of overtime [58]. CLASHBOT is a *non-embedded* agent [58]: with no game API, it reads the screen of an iPhone mirrored onto a Mac and clicks the mirrored window, the interface a human uses.

The task is hard for five measurable reasons. (i) *Real time*: perception targets 10 fps (7.97 fps in live run 111554); capture to click takes 323 ms (median, live trial). (ii) *Partial, noisy observation*: the opponent's deck, hand and elixir are hidden, and the pro's recorded move passes our perceived-legality check on only 0.542–0.597 of states. (iii) *Structured actions plus timing*: card × lane × zone or hold (D-010), and pros deploy within 1 s on about a fifth of seconds (dense act rate 0.204). (iv) *Sparse supervision*: 197 pro games (about 12 match-hours, one creator), no reward or simulator, an opponent with level-15 cards. (v) *Consumer hardware*: an Apple M5 laptop (32 GB), no paid API.

The design draws on the course's retrieval, knowledge-graph, RAG, recommendation and agent material [7, 23, 28, 30, 32] and follows a retrieve-then-rank, agent-with-tools pattern: deterministic tools (a tactical engine with a lane simulator, hybrid retrieval over pro states and a knowledge graph) propose at most four candidate plays with short text notes, and a decision LLM fine-tuned on the pro games picks one; no tool picks the move (D-045; decision log D-001 to D-053). Contributions:

1. A non-embedded, two-speed agent live on iPhone Mirroring, all models local, with guarded demo tools (Sec. 3).
2. A knowledge layer (hybrid retrieval over 10,923 pro states, a two-version KG with lane-simulation edges and 3,620 mined pro answers, a tactical engine) rendered as context for a fine-tuned setwise LLM ranker.
3. A leakage-checked benchmark suite (Sec. 4): options rebuilt after a leak (an option-key rule scored 0.852 on test), ten rankers compared with paired significance tests, ablations from retrieval to knowledge editing, and an open-loop comparison with the pro, negative results included.
4. Live Training Camp evaluation: 13 recorded runs (931 logged decisions) with one win, then a 0–10 trial on frozen code, diagnosed from the logs (Sec. 4.10, 5).

The bot plays only Training Camp and friendly battles,

never ranked matches against people (D-031, D-050).

2. Related Work

Game agents. AlphaStar [53] and OpenAI Five [3] reached expert level through game APIs and self-play, unavailable for a phone game. The closest work, KataCR [58], is a non-embedded Clash Royale agent (YOLOv8 detectors, OCR, an offline-RL StARformer policy in the Decision Transformer formulation [10]) that won 1 of 20 Training Camp games (Table 9); we reuse its detector weights [57] and protocol. Claude Royale [14] runs Claude Haiku agents at about 7 s per decision on the human ladder. *LLM agents and ranking.* ReAct, Reflexion and LATS [48, 64, 69] spend seconds per step on reasoning, reflection or search; a real-time budget rules out generation in the fast loop, so our LLM is a setwise reranker [49, 71] over tool proposals, as in two-stage recommender serving [30, 67]. Fine-tuning an LM to choose among items follows TALLRec and P5 [2, 13], graph facts in context follow GraphRAG [12], and when to consult the model, adaptive retrieval [16, 17].

3. System

Fig. 1 shows the two-speed design (D-007). CLASHBOT is an agent with tools [32], but unlike a ReAct agent [64] its LLM does not call them: the tools run on every decision, propose candidate plays and write notes, and only the fine-tuned d1 ranker picks one (D-045). An optional slow loop (Qwen3-14B, 4-bit [41]) serves as a long-horizon planner beside this fast actor [33]: it rewrites a three-line plan every 20 game seconds, never picks a move, and was used in one live run (21 plans, 5.98/6.86 s p50/p95). The default live path calls no generative LLM.

3.1. Perception and live state

Quartz window capture grabs the iPhone Mirroring window (632×1392 pixels, 22–38 ms per grab). Two YOLOv8 detectors [18] with KataCR's weights [57] (87.8 MB each) run on the M5 GPU (100.5 ms per call), RapidOCR [42] reads the clock and deploy labels in the background, a template matcher reads the hand slots (33.1 ms) only when a slot crop changes (D-038), and a Kalman tracker [19] follows units. The live state holds both sides' elixir, our hand, towers, the opponent's seen cards and each unit's time to reach a tower (ETA); ETA from card speed and attack range instead of the raw track cut its median absolute error from 1.49 s to 0.30 s and raised the share within 1 s from 34% to 80% (384 frames, 11 arrivals, one video). It also built the dense training data (D-040), so live and offline features match.

3.2. Data, knowledge graph and retrieval

Data. A scraper built a knowledge base from the wiki; 197 games from 33 videos of one professional creator were perceived and cleaned into 10,923 states (6,901 plays, 3,539 key moments, 483 periodic samples), with bad fields repaired and flagged, never deleted (D-011; 2,165 states lack a known placement). Splits are by video (SHA-1 order, 70/15/15), so no game spans splits (the contamination concern of RETRO [5]): train/val/test hold 23/5/5 videos, 142/27/28 games, 7,693/1,574/1,656 states and 3,641/761/766 ranker lines. A second pass at one state per second (∼17 h on the M5) gave 29,036/5,560/5,279 dense play/wait lines (does the pro deploy within 1 s?); 3,654 pro-advice documents complete the store.

Knowledge graph. Game knowledge is stored as a typed graph [7]. KG v1 (590 nodes, 12,030 edges, 12 relations) holds wiki COUNTERS and SYNERGY relations, deck membership and 129 mined ANSWERED_BY edges. KG v2 (D-048; 840 nodes, 27,112 edges, 19 relations) adds a game model: per-level stats for 126 cards (1,734 rows, opponents assumed at level 15), 8,951 STOPS, KILLS and PULLS edges from the lane simulator, 3,620 mined answer events (the opponent plays X; the pro's first defensive play within 8 s is Y) aggregated by support-counted rule mining [22, 37] into 287 rules with held rates, placement offsets and timing, 234 situation nodes, 450 PLAYED_AFTER sequences, deck co-occurrence, and role-similarity transfer for our cards without pro data; an opponent-deck model completes the hidden deck from the cards seen. Live, the graph supplies the engine's answer prior, pro placement offsets ($n \geq 5$), an elixir reserve for predicted air cards, a counter channel for candidates, and up to three `KG:` lines in d1's context, graph facts given to the ranker as in GraphRAG [12, 29] (Fig. 2).

Retrieval. Each state is a fixed-length, hand-built (not learned [26]) 644-d structured vector (3 scalars, 5 phase values, five 120-d card blocks for hand, deck, opponent deck and both sides' recent plays, 6 tower, 24 field and 6 threat values) plus a text rendering indexed by SQLite FTS5 BM25 [44]. Exact dot-product kNN and BM25 lists are fused by reciprocal rank fusion ($k = 60$) [11, 24], and the neighbours vote for the plays they contain. A 384-d bge-small text embedder channel [27, 59] and an approximate HNSW index [24, 38], compared with exact search, were evaluated offline only (Sec. 4.3).

3.3. Tactical engine, timing and the d1 ranker

Tactical engine. The engine (1,750 lines; roles for 122 cards) lists enemy pushes, estimates each push's tower damage by lane simulation over per-level stats, and proposes placed answers with reasons (a lone level-15 Balloon deals 3,162 tower HP unanswered, 351 with a Musketeer behind the tower, 0 with a Tesla pull). It values spells by kills (a Zap needs two kills and value 2.5), leads them by flight time, keeps buildings inside a pull box, and tracks elixir trades and offence windows; d1 reads its analysis as `tactics:`

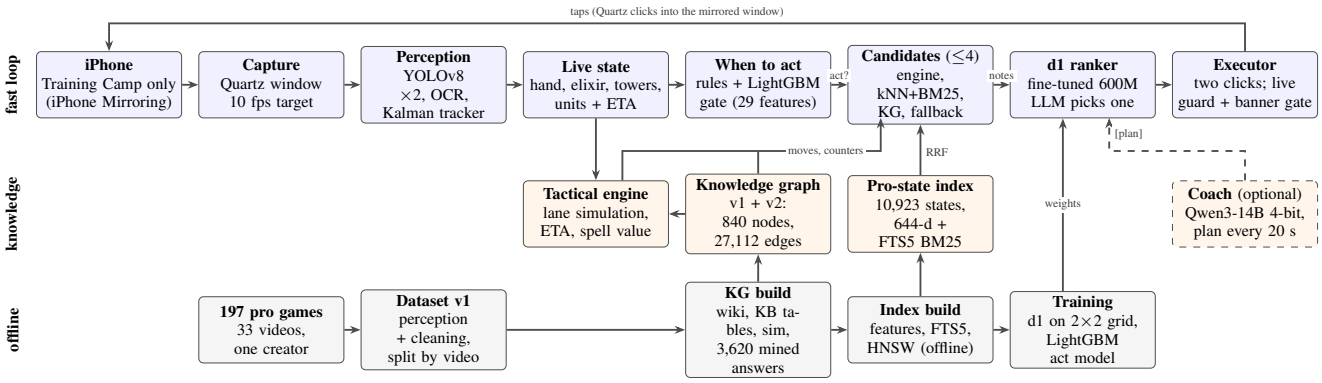


Figure 1. Two-speed architecture (D-007). The fast loop runs on every decision (323 ms p50 capture to click in the live trial); tools in the knowledge layer only propose candidates and write notes, and the fine-tuned d1 LLM picks the move (D-045). The coach is optional and off by default. Every decision is logged and can be replayed in the dashboard (Fig. 2).

lines and per-option notes (D-042).

When to act. A reactive policy (modes defend, leak, punish, counter, spell, save, none; D-041) holds by default and filters candidates by mode. A LightGBM [21] model on 29 features predicts whether a pro would deploy within 2 s (threshold 0.47, 0.05 ms per call); in the live *fast* policy rules override it in defend, counter, spell and leak, and save and none always hold. This gate is our adaptive trigger [16, 17, 29]: it decides when the LLM is consulted at all.

Candidates and d1. The generator returns at most four plays (up to two engine moves, then retrieval votes, KG v1 counter scores with weight 0.5, legal fallbacks). Hard symbolic constraints [9] then filter them: the card must be in the read hand with enough elixir and a legal placement, the policy's mode must allow the play, and fallbacks are held below 9.5 elixir. d1 is Liquid AI's d1-omni-600M [36], a 16-layer trunk whose head emits one logit per option marker, scoring all options in one pass (setwise LLM reranking [25, 49, 71]). Zero-shot it is at chance (Sec. 4.1), so, as TALLRec and P5 fine-tune a language model as a recommender [2, 13, 31], we fine-tuned the full text path (313.6M parameters, fp32 on MPS) with a softmax cross-entropy over the options: `d1-ft-joint` on states alone (3 epochs, 13,241 s), then the live `d1-ft-joint-tac` with tactics lines and notes (2 epochs, 4,554 s, peak 22.75 GB). Live, d1 takes 92.7/174.7 ms (p50/p95) and runs only when ≥2 candidates survive (119/279 plays, three pre-trial runs).

The option-set shortcut (D-037). Our first training set built three negatives per pro play (a neighbour's play, the same card elsewhere, another card at the same spot), so the positive was always the option sharing its card with one option and its placement with another: a rule reading only the option keys scored 0.848/0.853/0.852 on train/val/test. In the rebuilt 2×2 grid {pro card, second possible card} × {pro placement, second legal placement} every option shares its card and placement with exactly one other, so the key-only rule scores 0.25 by construction while the negatives stay hard (*cf*. RAFT's distractors [28, 68]); a random second placement removed a fixed lane order (lane-only rule 0.31 → 0.272).

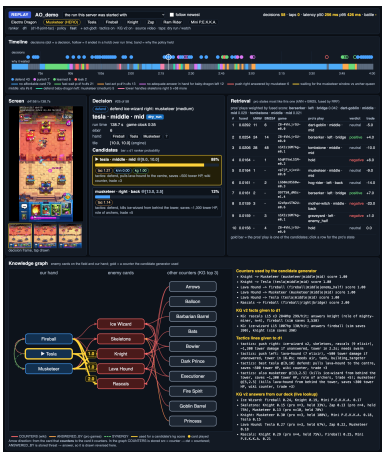


Figure 2. Decision dashboard on a video dry run. Top: timeline of decisions and hold reasons. Middle: decision frame; two candidates with d1 probabilities (Tesla 88%) and the engine, kNN and KG scores behind them; the ten pro states retrieved by kNN+BM25 with RRF. Bottom: KG subgraph (our hand, enemy cards, counters used) and the `KG:` and `tactics:` lines d1 read.

3.4. Executor, safety and demo tooling

The executor sends two calibrated Quartz clicks (card slot, then tile) 50 ms apart (execute stage 181 ms at the median in the trial), tracks own elixir, detects failed taps and cools a failed card down for 2 s. Two independent switches, the `--live` flag and `CLASHBOT_ALLOW_LIVE=1`, gate every tap, checked at construction and before every action (D-031). The demo launcher (about 10 s to load) taps only in battles whose opponent banner reads "Trainer"; an unreadable banner counts as not Training Camp (D-050). It read 411 of 412 recorded banners and 5 of 5 battles correctly. The dashboard (Fig. 2) replays or follows a run decision by decision; logging costs about 0.03 ms per decision.

Table 1. 4-way pick-the-pro-play accuracy (chance 0.25). "Notes" = tactics lines and per-option notes.

Model	Val	Test
Feature rankers, 783 features (seed 0)		
logistic regression	0.543	0.470
LightGBM LambdaRank	0.565	0.482
mean over 3 seeds	0.5484	0.4913
DCN-V2	0.537	0.403
d1-omni-600M decision LLM		
zero-shot	0.258	0.221
ft-joint, plain lines	0.524	0.483
ft-joint, given notes it never saw	0.477	0.461
ft-joint-tac (trained on notes; live)	**0.547**	**0.507**
ft-joint-kg (also trained on `KG:` lines)	0.530	0.484

4. Experiments

Setup. Offline numbers come from the frozen decision code (D-051, commit `66917e7`), with older outputs re-run. Ranker evaluations use 761 val and 766 test lines from 26 and 27 held-out games. Models are compared on the same lines; unless noted, p is an exact paired McNemar test, which treats lines as independent and so is optimistic. Thresholds, ensemble weights and feature-ranker configurations are chosen on val, so test is the clean column.

4.1. Ranking the pro's play

Table 1 gives the main result. The feature rankers score each option from 783 features with logistic regression, LightGBM LambdaRank [6, 21] and DCN-V2 [55], tuned on val, as a neural ranker should be compared with well-tuned simple baselines [30, 43]. (1) Zero-shot d1 is at chance: the 600M model has no usable Clash Royale knowledge, the parametric-memory limit of [28, 39]. (2) Fine-tuning lifts it 12.3 points above the placement prior on test (0.384, Table 2; $p < 0.001$). (3) The tuned feature ranker matches it: d1-ft-joint-tac minus LightGBM is +2.5 points (McNemar $p = 0.263$), and -0.3 against LightGBM's seed 2; LightGBM's top features by gain are the key's popularity (0.178) and $p(\text{zone} \mid \text{card})$ (0.124), the priors of the simple baselines. (4) Training on the notes makes d1 robust to them, not reliant on them: notes hurt a model that never saw them, and on those lines the note-trained model beats it by +4.6 points ($p = 0.006$), but its gain over the no-notes model on plain lines (+2.3, $p = 0.159$) is not significant, and deleting its notes does not lower its accuracy (Sec. 4.6). (5) Retraining with `KG:` lines did not help (-2.2 points): validation loss rose from 1.1916 to 1.6708 in epoch 2, and the KG facts on training lines are mined from the same games, a leak we have not removed.

By training frequency of the gold key (Fig. 3), popularity is the best model on head keys (0.696) and the worst on the tail (0.103), while d1-ft-joint-tac and LightGBM stay level across the two lower terciles (0.468/0.473, 0.457/0.458), generalising beyond memorised keys; a share of 0.14 of test gold keys never occurs in training.

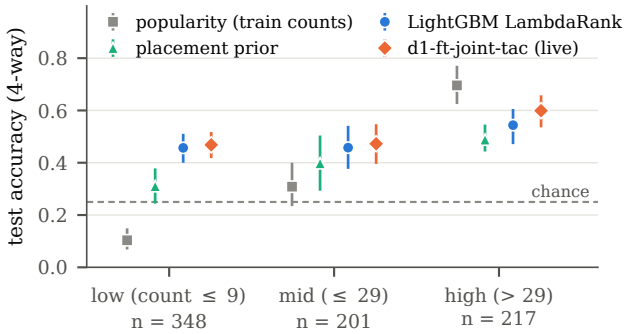


Figure 3. Test accuracy by training frequency of the gold key (terciles). Popularity wins on head keys and fails on the tail; the fine-tuned LLM and LightGBM hold up on rare keys. Data: `stats_audit_v1.json`.

Table 2. Decision models on the 766 test lines of Table 1 (d1's 0.5065 shows there as 0.507); middle block zero-shot. All differ from d1-ft-joint-tac at McNemar $p < 0.001$ but LightGBM ($p = 0.263$). †Thinking `between_tools`, ≤2,000 output tokens, answer parsed from the end; with the 16-token cap of the other API runs 62.8% of Sonnet replies were cut off (accuracy 0.144), 3.8% of Haiku's (counted wrong).

Model	Accuracy	Card	ms p50/p95	US$/dec.
d1-ft-joint-tac, 600M (live)	**0.506**	0.692	75/117	0
d1-2B fine-tuned, 1.9B	0.429	0.614	295/465	0
d1-omni-600M zero-shot	0.260	0.513	73/113	0
Qwen3-4B-Instruct-2507	0.286	0.527	802/1119	0
+ 3 retrieved pro states	0.283	0.516	1535/1958	0
Qwen3-14B	0.304	0.523	2721/3866	0
Claude Haiku 5.5	0.279	0.499	676/823	0.00009
Claude Sonnet 5.5†	0.364	0.585	2467/4824	0.00369
LightGBM LambdaRank	0.482	0.677	0.4/0.5	0
placement prior	0.384	0.614	0.003/0.003	0
popularity	0.325	0.453	0.001/0.002	0
random, one draw (exp. 0.250)	0.269	0.539	0.001/0.001	0

d1 is well calibrated (test ECE 0.048 over 15 bins; Brier 0.616): holding when its top probability is below 0.483 (chosen on val) answers 0.402 of test lines at 0.630 accuracy and raises the +1/0/−1 CRAG-style utility [63] from 0.013 to 0.104 (Fig. 4b). A d1+LightGBM ensemble changes test accuracy by -0.1, +2.1 or +3.9 points depending on LightGBM's seed, so we do not claim it; a rule arbiter narrowing d1's options when it is unsure gives no gain (0.509 vs 0.506) and stays off.

4.2. Decision models

Protocol (MAC-AU). The LLMs share one prompt: d1's input (state text with `tactics:` and `KG:` lines, options A–D with notes) and a request for the pro's choice. Qwen3 [41] decodes greedily, Claude Haiku with 16 output tokens, both thinking off (Sonnet: †); the RAG variant prepends the 3 nearest train-split pro states (600 characters each, plus the pro's play). Latency is wall time per decision after a warm-up (M5: d1 on MPS, Qwen in MLX 4-bit, one model at a

Table 3. Retrieval channels (916/902 val/test queries, train pool, depth 50). Hit@10: the pro's exact play is among the top-10 neighbours' plays.

List	hit@10 val / test	card hit@10 val / test	ms p50 (test)
BM25 (FTS5)	0.142 / 0.159	0.396 / 0.483	25.8
structured kNN (644-d)	0.166 / 0.136	0.434 / 0.412	2.1
text (bge-small, 384-d)	0.132 / 0.143	0.397 / 0.436	22.2
struct + BM25 (live)	0.138 / 0.160	0.434 / 0.487	28.0
all three, RRF	0.156 / 0.170	0.461 / 0.514	49.6

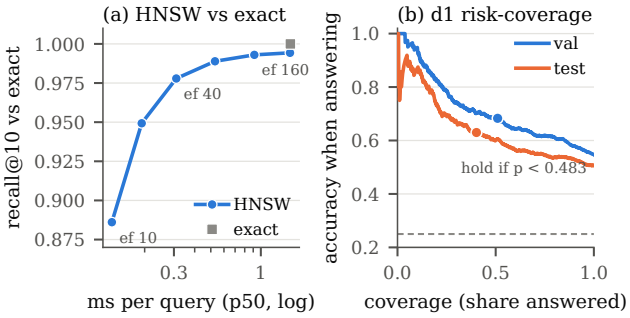


Figure 4. (a) HNSW recall@10 against exact kNN versus query time (10,923 vectors, k=10). (b) Risk–coverage of d1-ft-joint-tac holding below a probability threshold (dots: 0.483, set on val).

time; API: with the network); cost is usage times list price.

Findings. Fine-tuning dominates. Zero-shot, d1-omni-600M, both Qwen3 models and Claude Haiku 5.5 stay within 5.4 points of chance, the parametric-memory limit [28, 39] (d1: 0.260 with notes, 0.221 without, Table 1). Claude Sonnet 5.5, the best zero-shot model, is 14.2 points below the fine-tuned 600M at 33× its median latency and a per-decision cost: RankGPT-style zero-shot ranking [25, 49] falls short in this domain and budget. Three retrieved pro examples (RAG few-shot [28]) do not help Qwen3-4B (0.283 vs 0.286) and nearly double its latency. The larger fine-tune is worse: d1-2B (top 8 layers trained, val 0.4192) loses 7.7 points at 4× latency. LightGBM, a tuned baseline [30, 43], is on par at 0.4 ms.

4.3. Retrieval

In Table 3 the channels are within noise of each other (test hit@10 0.136–0.170): BM25 beats the structured channel on test (0.159 vs 0.136) but not on val, and the text channel adds about 20 ms for no clear gain, so the live list is structured plus BM25. As a lone candidate source, kNN voting loses to popularity on exact keys (test recall@10 0.317 vs 0.402) and wins only on cards (0.651 vs 0.607). HNSW at ef 40 keeps recall@10 at 0.978 in 0.31 vs 1.51 ms for exact search (Fig. 4a), but at 10.9k vectors exact search stays live. For pre- vs post-filtering [24], a deck pre-filter empties the pool for 88.9% of queries (decks unseen in training); post-filtering the top 10 returns nothing for 50.0% (deck) and 87.3% (phase) of queries, the top 200 for 0.5% and 0.1%.

Table 4. Held-out COUNTERS link prediction, test (762 queries, filtered, 3 seeds).

Method	MRR	H@1	H@10
random	0.0223	0.0113	0.1129
popularity	0.1103	0.0381	0.2385
TransE	0.0819	0.0000	0.2380
DistMult	0.0907	0.0232	0.1986
ComplEx	0.1162	0.0367	0.2616
RotatE	**0.1588**	0.0713	0.3259

4.4. KG embeddings and mined rules

Link prediction (Table 4). We hold out wiki COUNTERS triples (3,043/380/381) and rank 126 candidate cards (filtered, tie-aware); all four PyKEEN [1] models (TransE [4], DistMult [61], ComplEx [52], RotatE [50]) get the same grid (dimension 32 or 64, learning rate 0.01 or 0.001; chosen on val MRR) and three seeds, per the course's evaluation-audit protocol [7, 46, 51]. RotatE is best and beats popularity; TransE and DistMult do not, DistMult being symmetric by construction (asymmetry score 0.0) while only 940 of 3,804 counter pairs have a reverse edge.

Mined answer rules (E6; test in Table 5b). For each of 609/433 held-out val/test events we rank the pro's own deck (7.96/7.83 cards on average) with rules mined from training games (support $\geq$ 3, 8 s window); val hit@1: full 0.182, pro-mined only 0.191, wiki COUNTERS 0.117, train popularity 0.182, random 0.126. Pooled over val and test, the full score beats random (+0.044 hit@1) and the wiki counters (+0.043) but not the training popularity of the answer card (−0.006); on test the pro-mined source alone falls below popularity (−0.040 hit@1, bootstrap $p = 0.022$ over games). The score barely predicts whether the answer held the tower (AUROC 0.543), few rules have large support (10 or more for 63 of 1,042 events), and 8 s is the best window of 2, 4 and 8 s on val (MRR 0.409). Unlike AMIE and TLogic, which score rules on their mining data [22, 37], this held-out test against popularity [46] shows the 287 rules mostly re-encode the pro's card frequencies, a weak prior [9].

4.5. Retrieval methods for the KB and the KG

As retrievers (Table 5a) the hybrids lead (*cf*. Sec. 4.3 and [23–25]): RRF of all three lists has the best hit@10, card@10 and MRR, and the live dense+BM25 fusion beats both its lists by small margins (hit@10 0.175 vs 0.144, 0.166). HNSW keeps 0.977–0.998 of the exact top 20, but over 4,997 train states exact search takes only 1.16 ms (ef 10 saves 0.3 ms). The deck pre-filter returns nothing for 78.3% of lines (decks unseen in train; the pre-filter risk of [24]), though on the 16% it supports it is the most accurate KB rule (0.475). No method alone is a usable decision maker: the best, BM25, reaches 0.318, 18.9 points below d1, other KB methods 19–22 and the KG methods 25–29

Table 5. KB and KG methods on the 766 test lines, retrieving from train games. (a) KB, top 20: a hit's pro play has the gold key (hit@10, MRR) or card (card@10); rec. = overlap with exact top 20. (b) KG: 433 held-out threat→answer events, ranked over the pro's deck. 4-way: accuracy as a decision model (unsupported lines tied); Cov.: share of lines supported; ms: query p50. Two options share each card: a card-level score is capped at 0.5 4-way.

(a) KB method	hit@10	card@10	MRR	4-way	Cov.	ms
dense exact (644-d, dot)	0.144	0.423	0.067	0.284	0.53	1.16
BM25 (FTS5)	0.166	0.491	0.060	**0.318**	0.53	22.14
RRF dense + BM25 (live)	0.175	0.501	0.074	0.305	0.58	24.19
bge-small text (cosine)	0.155	0.450	0.067	0.307	0.46	13.64
RRF all three	**0.184**	**0.530**	**0.082**	0.314	0.54	45.16
HNSW ef 10 (rec. 0.977)	0.141	0.418	0.067	0.287	0.52	0.84
HNSW ef 40 (rec. 0.990)	0.141	0.419	0.067	0.286	0.53	1.11
HNSW ef 160 (rec. 0.998)	0.142	0.422	0.067	0.286	0.53	2.24
deck pre-filter, then dense	0.076	0.157	0.041	0.286	0.16	0.82
(b) KG method	**hit@1**	**hit@3**	**MRR**	**4-way**	**Cov.**	**ms**
v1 `counter_scores`	0.143	0.409	0.360	0.249	0.31	0.01
v1 wiki COUNTERS only	0.141	0.406	0.359	0.250	0.29	0.01
v2 `answers_for`, full	0.154	0.422	0.374	0.239	0.45	0.05
pro source only	0.129	0.399	0.350	0.236	0.20	0.04
role transfer only	0.145	0.398	0.356	0.241	0.24	0.04
wiki only	0.147	0.407	0.362	0.249	0.29	0.04
simulation (STOPS) only	0.157	**0.440**	0.376	**0.258**	0.31	0.04
KGE RotatE	0.125	0.400	0.340	0.253	0.49	0.18
KGE TransE	0.118	0.353	0.335	0.245	0.49	0.13
train popularity of the answer	**0.169**	0.433	**0.384**	0.217	0.49	0.00
random (expected)	0.128	0.384	0.345	0.250	–	–

Table 6. Context conflict on d1-ft-joint-tac (E2), test lines (n = 766, 27 games); p = exact McNemar; follow-rate as in the text (C4: 12 movable lines).

Condition	Accuracy	Δ pts (p)	Changed	Follow
C0 as exported	0.507	–	–	–
C1 context removed	0.508	+0.1 (1.00)	21%	–
C2 notes shuffled	0.517	+1.0 (0.27)	7%	0.094
C3 notes poisoned	0.503	−0.4 (0.76)	6%	0.044
C4 block transplanted	0.495	−1.2 (0.45)	19%	0.083

points below (all McNemar $p < 10^{-12}$), as a card-level score cannot place and finds a threat on 374 of 766 lines; no KG method beats train popularity on hit@1 or MRR (0.169, 0.384). Their value is as candidate sources and ranker context (Sec. 5).

4.6. Context conflict: does d1 follow notes? (E2)

Following ClashEval and the knowledge-conflict lecture [34, 56, 60], we corrupt d1-ft-joint-tac's context with the gold label fixed: C1 removes the notes and `tactics:/KG:` lines; C2 shuffles the notes across options; C3 swaps the gold option's informative note with a distractor's and names a wrong card in the `tactics:` line (relevant but misleading); C4 transplants another line's tactics/KG block (irrelevant). Follow-rate: the share of movable lines (the corruption favours an option d1 did not pick) whose pick moves to the favoured option.

As Table 6 shows, no condition moves accuracy by more than 1.2 points. Removing the whole context changes 21% of picks, but gains and losses cancel (57 vs 56 lines); poisoned notes move 4.4% of movable picks. Lines whose gold option carries the informative note are easier than lines where a distractor's does (0.564 vs 0.371), but the gap persists with no notes (0.599 vs 0.401), so it reflects which states are easy. Context-aware decoding [47] selects weight $a = 0$ on val in every condition, and each $a > 0$ lowers C0 test accuracy by 1.7 to 2.9 points; AdaCAD's adaptive weight [54] averages 0.014 and changes accuracy by −0.1 points. d1 thus avoids both failures the course warns about, adopting wrong context over its prior (ClashEval [56]) and greater sensitivity to relevant-but-misleading than to irrelevant noise (RAGChecker [45]), because it barely reads its notes, plausibly as they favour the gold option on only 30% of test lines and a distractor on 22% (49% have none), so it is robust to wrong tool output.

4.7. When to act

On dense test seconds (n = 5,279), balanced accuracy for "the pro deploys within h" at h = 1/2/3 s is 0.506/0.509/0.510 for the hand-written rules and 0.594/**0.607**/0.613 for LightGBM on 29 features (AUC 0.632/0.659/0.672); the live *fast* combination, with rules overriding the model in four modes, sits between them (0.571/0.584/0.593). On the balanced subset (h = 1 s, n = 3,221) used to train a d1 act head (top 8 layers, 168.9M trainable parameters), LightGBM scores 0.598 against the head's 0.558, at 0.05 ms instead of about 70 ms per call: again the tuned simple model beats the LLM.

4.8. Next card, opponent deck, knowledge editing

For opponent next-card prediction, a sequential recommendation task [20, 31], SASRec (3 seeds) reaches acc@1 0.129/0.222 on val/test and loses to a copy-frequency rule (0.207/0.243), the copy intuition of CyGNet [8, 70]; popularity is near zero (0.000/0.002) because opponents vary, and the ceiling set by cards already seen is 0.678/0.644. The KG opponent-deck model recovers hidden cards with recall 0.234/0.291/0.327 after 1/2/3 seen cards, against 0.130/0.124/0.118 for popularity (leave-one-game-out, 155 games). For knowledge editing [34, 66] we apply a synthetic +1-elixir patch to five cards in a patch-versioned fact store, SERAC's non-parametric route [40], which injects new knowledge without retraining [35]: reliability, generalisation and portability are 1.000 (0.000, 0.000, 0.067 without the edit), locality 1.000, rollback passes, and reweighting cuts retrieved hits using an edited card from 0.124 to 0.002; the store is deterministic, so this shows the mechanism, not that it generalises to a real patch [62].

4.9. Bot versus pro on held-out videos

For a trajectory-level evaluation [65], we run the live stack open loop on held-out test videos with the pro's deck and

Table 7. Bot vs pro on held-out pro videos, open loop: 24 test games, 1,561 pro plays. Percent; window ±2 s. Null: the bot's own taps shifted by 20–60 s. Baselines use the same plays; the placement prior is given the pro's card.

Measure	Bot	Baseline
pro plays met by a bot tap	77.1	68.9 (null)
bot taps met by a pro play	72.6	67.9 (null)
taps (bot) / plays (pro) per min	22.8	17.4
same card, matched pairs	17.1	12.8 (random in hand)
same card, KG threat exists (126)	19.8	15.6 (KG v1 counter)
pro's card in perceived hand	40.1	53.0 (dataset's hand)
same lane | same card (103)	42.7	44.6 (placement prior)
same key | same card (103)	26.2	29.7 (placement prior)

Table 8. (a) Latency by stage, ms p50 / p95: live trial session (363 decisions) and mean of 3 video dry runs (a shared Mac, so upper bounds). (b) Live record in Training Camp.

(a) Stage	trial session	dry run, 3 reps
capture	22.0 / 44.8	0.8 / 4.3
perception (detector, OCR, hand)	108.0 / 266.4	179.2 / 234.2
live state	0.13 / 0.31	0.2 / 0.5
trigger (rules + LightGBM)	1.1 / 9.2	2.7 / 9.7
candidates (engine, retrieval, KG)	26.8 / 57.9	46.7 / 86.1
rank (d1; 0 when one candidate)	0.0 / 140.9	87.1 / 191.0
execute (two clicks)	181.0 / 188.8	no clicks
total	**322.7 / 601.6**	271.7 / 455.2

(b) Live record	
runs before the trial, 11-03 to 11-07	13 (12 live), 931 decisions, 746 taps
taps landed, two newest of them	75/84, 78/95 (older run 233907: 95/100)
run 013441 (3 battles)	tower taken; 3-crown loss; **win in overtime**
trial: wins / battles	**0 / 10**
crowns for / against (10 battles)	3 / 18; a tower taken in 3; 2 in overtime
per logged battle (9 of 10)	40.3 decisions, 27.0 taps; median 213 s

a dry-run executor, matching its would-be taps to the pro's plays within ±2 s. As taps do not change the video, this measures what the bot would do in the pro's place, not whether it would win (its hand memory never learns a card left the hand). Retrieval uses only train and val games, the KG answers and placement prior only train games. Results cover the 24 usable test games (90 min, 1,561 pro plays, 2,042 bot taps); 4 more lack a full deck.

Table 7 shows that the bot's timing beats the null (matched taps a median 0.47 s from the pro's play), which matters because it taps more often and spends more elixir than the pro (58.7 vs 33.6 per minute). Its card choice beats a random card of its hand and, where a KG threat exists, the KG v1 counter rule, but the ceiling is low: the pro's card is in the perceived hand on only 40.1% of plays, and the bot cannot pick a card it does not see. Missed pro plays were mostly held because no card was affordable (23.0%), the engine left the push to the tower (22.7%) or the act model waited (18.8%).

4.10. Live play and prior bots

The 13 runs before the trial (Table 8b; 12 live, one dry) took 347/613 ms p50/p95 in total (339/572 ms in run 111554). In the trial (Table 8a) perception (108 ms p50) and the two-click execute stage (181 ms) dominate; d1 adds nothing at the median, as most decisions have one candidate, and 141 ms at p95. The 300 ms p95 target was missed in every live run and all three dry runs. Failed taps concentrate on Tesla (11 of 19 landed in run 111554) and Fireball (3 of 7). *Trial.* On 2026-11-07 ten Training Camp battles were played through the launcher (`scripts/demo.sh --live`) on the frozen code (D-051), results recorded by hand. All ten were lost (Table 8b): 0–1 five times (once in overtime), 0–2, 0–3, 1–2, and 1–3 twice (once in overtime); against KataCR's 1/20 a two-sided Fisher exact test gives $p = 1.0$. The session log holds 9 of the 10 battles (one start was missed), and the banner gate correctly refused two non-battle screens whose banner read "2618".

Table 9. Prior Clash Royale agents and CLASHBOT. R = reported by the authors, M = measured by us. Protocols differ; see text.

	KataCR [58]	Claude Royale [14]	CLASHBOT (ours)
Interface	video stream, ADB taps (Linux)	BlueStacks (macOS)	iPhone Mirroring, Quartz clicks (macOS)
Perception	YOLOv8-1×2 (mAP 67.4), OCR, ResNet classifiers	Claude vision	KataCR's YOLOv8 weights, OCR, Kalman tracker
Decision	offline-RL StARformer	Claude Haiku, 3 agents	gate, engine, retrieval, KG; fine-tuned 600M LLM picks
Knowledge used	none (in weights)	hand-written prompts, notes	10,923 pro states, KG v1/v2, tactics engine
Training data	105 episodes, 113,981 frames, one player vs AI	none (prompting)	197 pro games, 33 videos
Latency	120 ms decision + 240 ms fusion (R)	∼7 s per agent (R)	323/602 ms p50/p95, capture to click, trial (M)
Offline metric	none	card choice 11/25 on labelled screenshots (R)	4-way test accuracy 0.507 (M)
Opponent	built-in AI (Training Camp)	human ladder	built-in AI (Training Camp)
Win rate	1/20 = 5%; 0–10% over 9 models (R)	none (ladder; not comparable)	trial 0/10; one earlier win (M)

Only KataCR and CLASHBOT report a win rate against the built-in AI (Table 9), and under different protocols: KataCR played the 2024 game (Season 59) with another deck, and its 1/20 is the best of ten epochs selected on the same 20 games (optimistic); its latency excludes actuation, ours includes it. We reuse its detector and claim no better one.

5. Discussion and Limitations

Four results are negative, and each repeats a course lesson. *Tuned simple baselines match the LLM* on ranking and timing (Sec. 4.1, 4.2, 4.7), and copy-frequency beats SASRec: well-tuned simple models often match neural ones [30, 43], and 197 games from one creator leave a 600M model little to learn beyond the priors a GBDT reads directly. *d1 does not read its context* (Sec. 4.6), the opposite extreme from the over-trusting models of [45, 56]; with the engine near chance as a ranker (4-way 0.293/0.265 val/test), the tools contribute by proposing candidates, not by writing

notes [30]. *Mined rules approximate popularity* (Sec. 4.4): the evaluation audits the course teaches [46, 51] ask for exactly this baseline (the protocol matters as much as the method), and the same habit caught the option-set leak before d1 was fine-tuned on it (D-037). *Perception caps agreement with the pro*: besides the 40.1% hand ceiling and the 0.542–0.597 legality rate, in an early live run 14 of 27 decisions were skipped because the hand read as unknown. As the ESCA lecture argues, perception errors dominate embodied-agent failures [8, 15].

Why the trial was lost (limitations and future work). (1) *Tower HP:* OCR drops digits (one tower read 4424↔424 seven times in 30 s) and a fallen tower reads unknown, not 0, so the deploy area it opens went unused (a tower fell in 3 battles). (2) *No committed target:* counter-pushes follow the defended push's lane, splitting damage (one battle ended with enemy towers at 1,663 and 1,824 HP, neither taken), and after a tower fell ram riders kept taking its lane, *i.e.* went at the king. (3) *No support* troop is offered behind a surviving defence or a ram rider. (4) *Spell value:* Fireball is valued by elixir destroyed, not tower damage prevented, so none was proposed against a bunched Giant, Balloon and Bats push, and the Zap group rule counts tanks. (5) *Over-playing:* 27 taps per battle, 40 emergency defends at 2–5 elixir and 30 tap pairs under 1.6 s apart, as on video (Table 7). (6) *Phantom threats:* a stationary "Graveyard", a card the trainer's deck lacks, triggered defences six times. All six sit in the hand-written engine and policy, which our benchmarks rate lowest (timing rules 0.509 balanced accuracy, the engine 0.265 as a ranker), and d1 cannot repair them: it ranks only when two or more candidates survive (119 of 279 executed plays) and ignores the notes. Fixes (monotone HP filter, per-battle target tower, pocket placements, support moves, spell value from prevented damage, elixir budget at the pro's rate) are specified for a re-run against this 0/10 baseline.

Other limitations. d1's serve-time context carries a `[plan]` line no training line contains, an unmeasured train/serve mismatch. Every offline number rests on 26–27 held-out games from one creator, and three of our eight cards have no pro plays, so the engine and role transfer carry them. Video evaluation is open loop; perception ignores enemy health bars (units are simulated at full HP). Trial results were recorded by hand: the end-of-battle frames missed the result screen in 5 of 9 battles.

6. Conclusion

CLASHBOT shows that a non-embedded agent can play Clash Royale live on a laptop, with knowledge-graph, retrieval and simulation tools feeding a fine-tuned LLM with the final say, at 323 ms from capture to click. Fine-tuning turned a chance-level LLM into one that picks the pro's play half the time, but tuned simple models match it, it ignores the tools' notes, and the agent does not yet beat Training Camp reliably (one early win, then 0/10). No zero-shot LLM comes within 14 points of the fine-tuned d1 (Claude Sonnet 5.5 36.4%, Qwen3-14B 30.4%, Claude Haiku 5.5 27.9%; chance 25%), nor any KB or KG method alone within 18. Next steps: fix the trial failures of Sec. 5 and re-run the 10-battle trial with automatic result logging, measure perception against labelled frames, and train d1 on lines where notes and leak-free KG facts carry signal the state lines do not. Code, tests, benchmarks (`docs/data/BENCHMARKS.md`) and the decision log are in the repository.

References

[1] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, Mikhail Galkin, Sahand Sharifzadeh, Asja Fischer, Volker Tresp, and Jens Lehmann. Bringing light into the dark: A large-scale evaluation of knowledge graph embedding models under a unified framework. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. 5

[2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *ACM Conference on Recommender Systems (RecSys)*, 2023. arXiv:2305.00447. 2, 3

[3] Christopher Berner, Greg Brockman, Brooke Chan, et al. Dota 2 with large scale deep reinforcement learning. arXiv:1912.06680, 2019. 2

[4] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013. 5

[5] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2112.04426. 2

[6] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010. 4

[7] CAP 6938 Course Staff. Lecture 5: Knowledge graphs. Course lecture slides, CAP 6938, University of Central Florida, 2026. 1, 2, 5

[8] CAP 6938 Course Staff. Lecture 6: Temporal and visual knowledge graphs. Course lecture slides, CAP 6938, Topic 02C, University of Central Florida, 2026. 6, 8

[9] CAP 6938 Course Staff. Lecture 7: Symbolic and neuro-symbolic reasoning. Course lecture slides, CAP 6938, University of Central Florida, 2026. 3, 5

[10] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021. 2

[11] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet

and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, 2009. 2

[12] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. 2

[13] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *ACM Conference on Recommender Systems (RecSys)*, 2022. arXiv:2203.13366. 2, 3

[14] Ethan Houseworth. Claude Royale: AI agents playing Clash Royale autonomously. GitHub repository, https://github.com/houseworthe/claude-royale, 2025. Created 2025-12-07; last commit 2026-01-25 (70ecc8c); MIT licence; accessed 2026-11-07. 2, 7

[15] Jiani Huang, Amish Sethi, Matthew Kuo, Mayank Keoliya, Neelay Velingker, JungHo Jung, Ser-Nam Lim, Ziyang Li, and Mayur Naik. ESCA: Contextualizing embodied agents via scene-graph generation. In *NeurIPS*, 2025. 8

[16] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2403.14403. 2, 3

[17] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.06983. 2, 3

[18] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics, 2023. Version 8.0.0; AGPL-3.0. 2

[19] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82 (1):35–45, 1960. 2

[20] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *IEEE International Conference on Data Mining (ICDM)*, 2018. arXiv:1808.09781. 6

[21] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 3146–3154, 2017. 3, 4

[22] Jonathan Lajus, Luis Galárraga, and Fabian Suchanek. Fast and exact rule mining with AMIE 3. In *ESWC*, 2020. 2, 5

[23] Ser Nam Lim. Retrieval I: Representation. Lecture 08, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Sep. 29, 2026. 1, 5

[24] Ser Nam Lim. Retrieval II: Indexing and search. Lecture 09, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Oct. 1, 2026. 2, 5

[25] Ser Nam Lim. Retrieval III: Ranking and generative retrieval. Lecture 10, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Oct. 6, 2026. 3, 5

[26] Ser Nam Lim. Dense knowledge representation I: Fixed-length vectors, the objectives that shape them, and the training choices that matter. Lecture 02, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Sep. 1, 2026. 2

[27] Ser Nam Lim. Dense knowledge representation II: Unified multimodal embedders, and DINO self-supervised vision, v1 to v3. Lecture 03, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Sep. 3, 2026. 2

[28] Ser Nam Lim. Retrieval-augmented generation I: Architectural formulations, joint pretraining & fine-tuning dynamics, contextual mechanics, and rigorous evaluation frameworks. Lecture 11, Topic 05, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 1, 2026. 1, 3, 4, 5

[29] Ser Nam Lim. Retrieval-augmented generation II: Dynamic retrieval triggers, reinforcement learning for autonomous search agents, and graph-structured indexing architectures. Lecture 12, Topic 05, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 6, 2026. 2, 3

[30] Ser Nam Lim. Recommendation I: Collaborative filtering, ranking losses, and offline evaluation. Lecture 13, Topic 06, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 8, 2026. 1, 2, 4, 5, 7, 8

[31] Ser Nam Lim. Recommendation II: Sequential and generative recommenders. Lecture 14, Topic 06, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 8, 2026. 3, 6

[32] Ser Nam Lim. AI agents I: Tools, reasoning loops, planning, and context management. Lecture 15, Topic 07, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 20, 2026. 1, 2

[33] Ser Nam Lim. AI agents II: Memory, long-horizon agents, and multi-agent systems. Lecture 16, Topic 07, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 22, 2026. 2

[34] Ser Nam Lim. Knowledge updating I: Model editing and knowledge conflicts. Lecture 17, Topic 08, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 27, 2026. 6

[35] Ser Nam Lim. Knowledge updating II: Continual learning and knowledge injection. Lecture 18, Topic 08, CAP 6938 AI Knowledge Systems, University of Central Florida, 2026. Lecture slides, October 29, 2026. 6

[36] Liquid AI. d1-omni-600M. Hugging Face model card, https://huggingface.co/LiquidAI/d1-omni-600M, 2026. LFM Open License; accessed 2026-11-07. 3

[37] Yushan Liu, Yunpu Ma, Marcel Hildebrandt, Mitchell Joblin, and Volker Tresp. TLogic: Temporal logical rules for explainable link forecasting on temporal knowledge graphs. In *AAAI*, 2022. 2, 5

[38] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. arXiv:1603.09320. 2

[39] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2212.10511. 4, 5

[40] Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D. Manning, and Chelsea Finn. Memory-based model editing at scale. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2206.06520. 6

[41] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025. 2, 4

[42] RapidAI. RapidOCR: An OCR toolkit based on ONNXRuntime. GitHub repository, https://github.com/RapidAI/RapidOCR, 2021. Package rapidocr_onnxruntime; accessed 2026-11-07. 2

[43] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. Neural collaborative filtering vs. matrix factorization revisited. In *ACM Conference on Recommender Systems (RecSys)*, 2020. arXiv:2005.09683. 4, 5, 7

[44] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. 2

[45] Dongyu Ru, Lin Qiu, Xiangkun Hu, et al. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. arXiv:2408.08067. 6, 7

[46] Daniel Ruffinelli, Samuel Broscheit, and Rainer Gemulla. You CAN teach an old dog new tricks! on training knowledge graph embeddings. In *ICLR*, 2020. 5, 8

[47] Weijia Shi, Xiaochuang Han, Mike Lewis, et al. Trusting your evidence: Hallucinate less with context-aware decoding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2305.14739. 6

[48] Noah Shinn, Federico Cassano, Ashwin Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366. 2

[49] Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2304.09542. 2, 3, 5

[50] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. Rotate: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019. 5

[51] Zhiqing Sun, Shikhar Vashishth, Soumya Sanyal, Partha Talukdar, and Yiming Yang. A re-evaluation of knowledge graph completion methods. In *ACL*, 2020. 5, 8

[52] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction. In *ICML*, 2016. 5

[53] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. 2

[54] Han Wang, Archiki Prasad, Elias Stengel-Eskin, and Mohit Bansal. AdaCAD: Adaptively decoding to balance conflicts between contextual and parametric knowledge. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025. arXiv:2409.07394. 6

[55] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the Web Conference (WWW)*, 2021. arXiv:2008.13535. 4

[56] Kevin Wu, Eric Wu, and James Zou. ClashEval: Quantifying the tug-of-war between an LLM's internal prior and external evidence. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2404.10198. 6, 7

[57] Tianyang Wu. KataCR: Non-embedded Clash Royale agent (code, detection dataset and YOLOv8 weights). GitHub repository, https://github.com/wty-yy/katacr, 2024. Commit 36ceb9f; MIT licence; detector weights v0.7.13; accessed 2026-11-07. 2

[58] Tianyang Wu, Lipeng Wan, Yuhang Wang, Qiang Wan, and Xuguang Lan. Playing non-embedded card-based games with reinforcement learning. arXiv:2504.04783, 2025. Also in ICIRA 2024, LNCS vol. 15206, pp. 255–269, doi 10.1007/978-981-96-0792-1_20. 1, 2, 7

[59] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. arXiv:2309.07597, 2023. 2

[60] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.13300. 6

[61] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *ICLR*, 2015. 5

[62] Wanli Yang, Fei Sun, Jiajun Tan, et al. The mirage of model editing: Revisiting evaluation in the wild. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.11177. 6

[63] Xiao Yang, Kai Sun, Hao Xin, et al. CRAG: Comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. arXiv:2406.04744. 4

[64] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629. 2

[65] Shunyu Yao, Noah Shinn, Pedram Razavi, et al. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.12045. 6

[66] Yunzhi Yao, Peng Wang, Bozhong Tian, et al. Editing large language models: Problems, methods, and opportunities. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.13172. 6

[67] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *ACM Conference on Recommender Systems (RecSys)*, 2019. 2

[68] Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. RAFT: Adapting language model to domain specific RAG. *arXiv preprint arXiv:2403.10131*, 2024. 3

[69] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, et al. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2310.04406. 2

[70] Cunchao Zhu, Muhao Chen, Changjun Fan, Guangquan Cheng, and Yan Zhang. Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks. In *AAAI*, 2021. 6

[71] Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. A setwise approach for effective and highly efficient zero-shot ranking with large language models. In *Proceedings of the 47th International ACM SIGIR Conference*, 2024. arXiv:2310.09497. 2, 3